API Development Documentation

A Secure and Scalable Node.js API Architecture

Prepared by: xAI

Date: August 14, 2025

Contents

1 Purpose

This document outlines the architecture for a secure, scalable, and maintainable Node.js API built with Express and MongoDB, adhering to modern best practices. It emphasizes robust security measures (rate limiting, password hashing, JWT authentication), optimized performance through database indexing, and maintainable code via centralized error handling and a modular folder structure.

2 Folder Structure

```
project/
 src/
    config/
       db.js
    controllers/
       authController.js
    middlewares/
       asyncHandler.js
       errorHandler.js
       validateRequest.js
       rateLimiter.js
    models/
       User.js
    routes/
       authRoutes.js
    utils/
       logger.js
       generateToken.js
    validations/
        authValidation.js
 .env
 package.json
 server.js
```

3 Dependencies

The following dependencies are required and can be installed using:

```
npm install express mongoose dotenv bcrypt jsonwebtoken joi
   express-rate-limit morgan winston
```

4 Database Setup

4.1 src/config/db.js

```
const mongoose = require("mongoose");

```

```
const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log(" MongoDB Connected");
  } catch (error) {
    console.error(" Database connection failed", error);
    process.exit(1);
  }
};

module.exports = connectDB;
```

5 Environment Variables

5.1 .env

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/myapp
JWT_SECRET=supersecretkey
JWT_EXPIRES_IN=1d
```

6 Middleware

6.1 Logging

6.1.1 src/utils/logger.js

```
const morgan = require("morgan");
module.exports = morgan("dev");
```

6.2 Rate Limiting

6.2.1 src/middlewares/rateLimiter.js

```
const rateLimit = require("express-rate-limit");

module.exports = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
  message: "Too many requests, try again later",
});
```

6.3 Request Validation (Joi)

6.3.1 src/middlewares/validateRequest.js

```
module.exports = (schema) => {
  return (req, res, next) => {
    try {
      const { error } = schema.validate(req.body, { abortEarly:
         false });
      if (error) {
        return res.status(400).json({
          status: "error",
          errors: error.details.map((err) => err.message),
        });
      }
      next();
    } catch (err) {
      next(err);
    }
  };
};
```

7 Error Handling

7.1 src/middlewares/errorHandler.js

```
module.exports = (err, req, res, next) => {
  console.error(err.stack);
  res.status(err.statusCode || 500).json({
    status: "error",
    message: err.message || "Internal Server Error",
  });
};
```

8 User Schema with Indexes

8.1 src/models/User.js

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, index:
     true },
  password: { type: String, required: true },
}, { timestamps: true });

module.exports = mongoose.model("User", userSchema);
```

9 Joi Validation Schema

9.1 src/validations/authValidation.js

```
const Joi = require("joi");

exports.registerSchema = Joi.object({
  name: Joi.string().min(3).max(50).required(),
  email: Joi.string().email().required(),
  password: Joi.string().min(6).required(),
});

exports.loginSchema = Joi.object({
  email: Joi.string().email().required(),
  password: Joi.string().required(),
});
```

10 Authentication (JWT)

10.1 src/utils/generateToken.js

```
const jwt = require("jsonwebtoken");

module.exports = (id) => {
  return jwt.sign({ id }, process.env.JWT_SECRET, {
    expiresIn: process.env.JWT_EXPIRES_IN,
  });
};
```

11 Password Hashing

Password hashing is implemented using bcrypt within the controller before storing user data in the database.

12 Example API Routes

12.1 src/routes/authRoutes.js

```
const express = require("express");
const { register, login } = require("../controllers/
   authController");
const validateRequest = require("../middlewares/validateRequest")
   ;
const { registerSchema, loginSchema } = require("../validations/
   authValidation");

const router = express.Router();

```

```
router.post("/register", validateRequest(registerSchema),
   register);
router.post("/login", validateRequest(loginSchema), login);

module.exports = router;
```

13 Centralized Async Handler

13.1 src/middlewares/asyncHandler.js

```
module.exports = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};
```

14 Controller with Try-Catch

14.1 src/controllers/authController.js

```
const bcrypt = require("bcrypt");
const User = require("../models/User");
const generateToken = require("../utils/generateToken");
const asyncHandler = require("../middlewares/asyncHandler");

exports.register = asyncHandler(async (req, res) => {
  try {
    const { name, email, password } = req.body;
    const userExists = await User.findOne({ email }).lean();
    if (userExists) return res.status(400).json({ message: "User
       already exists" });

    const hashedPassword = await bcrypt.hash(password, 10);
    const user = await User.create({ name, email, password:
       hashedPassword });

    res.status(201).json({ token: generateToken(user._id) });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

exports.login = asyncHandler(async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email }).lean();
    if (!user) return res.status(400).json({ message: "Invalid
       credentials" });

    const match = await bcrypt.compare(password, user.password);
```

```
    if (!match) return res.status(400).json({ message: "Invalid
       credentials" });

    res.json({ token: generateToken(user._id) });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});
```

15 Checklist

Environment variables managed via .env file

Rate limiting implemented for API security

Centralized error handling for consistent responses

Async handler for streamlined controller logic

Request validation using Joi

Password hashing with bcrypt

JWT-based authentication

Database indexing for query performance

Logging with Morgan and Winston

Modular and scalable folder structure

16 Security Considerations

- Always hash passwords before storage using bcrypt.
- Enforce HTTPS in production environments.
- Securely store JWT secrets in environment variables.
- Implement Cross-Origin Resource Sharing (CORS) restrictions.
- Utilize the `helmet` package to set secure HTTP headers.